Check for updates

Internet Technology Letters

WILEY

SPECIAL ISSUE ARTICLE OPEN ACCESS

TinyFed: Lightweight Federated Learning for Constrained IoT Devices

Claudio Jr. N. da Silva | Maycon L. M. Peixoto | Gustavo B. Figueiredo | Cassio V. S. Prazeres

Institute of Computing, Federal University of Bahia, Salvador, Brazil

Correspondence: Cassio V. S. Prazeres (prazeres@ufba.br)

Received: 26 April 2025 | **Revised:** 13 May 2025 | **Accepted:** 9 June 2025

Keywords: constrained devices | IoT | machine learning | neural networks | TinyFL

ABSTRACT

TinyML enables machine learning inference on microcontrollers with limited resources. Extending this to a collaborative setting led to Tiny Federated Learning (TinyFL). This article presents TinyFed, a lightweight framework that supports the full federated learning cycle—from local training to model aggregation and redistribution. TinyFed was validated on ESP32 devices using a neural network with four inputs, three hidden layers, and two outputs to detect temperature, humidity, luminosity, and voltage anomalies. Local training achieved accuracies of up to 99.47%.

1 | Introduction

Integrating the Internet of Things (IoT) and Machine Learning (ML) has become a significant research topic in networking, as it leverages sensors to gather data for ML model execution on a central server. However, challenges such as bandwidth limitation, latency constraints, privacy concerns, and data silos hinder complete integration [1]. To address these issues, Fog/Edge Computing and Federated Learning (FL) bring intelligence closer to the edge, and consequently to the end user, reducing latency and bandwidth needs [2].

An additional component in the advancement of the integration of IoT and ML is the TinyML framework. It enables ML models to operate on resource-constrained edge devices such as microcontrollers, despite the limitations in processing, memory, and power [3] capacities.

Researchers are expanding TinyML's capabilities by integrating FL, resulting in Tiny Federated Learning (TinyFL). In this approach, resource-constrained devices train models locally and collaborate by sharing model updates, as in traditional FL [4, 5]. In TinyFL, model weights are periodically sent to a central server for aggregation, disseminating an updated global model to all participating clients.

The TinyFL framework increases processing, memory, and energy demands, as devices perform local training, inference, and model exchange with a central server [6]. Research in this area remains emerging [7]. Wulfert et al. [8] and Muhammed and Ibraheem [9] highlight challenges in deploying FL on constrained IoT devices, while Ficco et al. [10] explore hardware configurations. Llisterri et al. [11] examine local training nuances. Most studies address isolated aspects, but none deliver a full FL cycle with result evaluation while maintaining data collection on the device.

This article presents a lightweight solution for sensor data collection, local storage, on-device ML training, and weight sharing among devices. Unlike related works focused on specific tasks, our approach emphasizes processing optimization and evaluates FL performance on constrained IoT devices. We deploy three ESP32 microcontrollers to read temperature and humidity in a controlled environment. Each device trains a neural network

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). *Internet Technology Letters* published by John Wiley & Sons Ltd.

with four input neurons, three hidden layers (16, 8, and 4 neurons), and a two-neuron output layer for anomaly detection.

2 | Related Work

As defined by [12], TinyFL refers to deploying FL on low-power, resource-constrained devices, enabling real-time ML tasks—such as processing, analysis, and inference—at the edge. It is designed for environments where conventional ML models are impractical due to hardware limitations.

Wulfert et al. [8] and Muhammed and Ibraheem [9] examine the challenges of implementing FL on constrained IoT devices, including privacy risks, centralized server dependence, and communication overhead. Wulfert et al. propose gossip learning for decentralized communication, while Muhammed and Ibraheem introduce Alpha-FedAvg, though its implementation details remain unspecified. Dai and Tan [13] propose an FL method optimized for computational and communication efficiency on microcontrollers, emphasizing the challenges of deploying FL on low-power devices.

Aatab and Freitag [14] present a neural network library tailored for FL, improving IoT device adaptability in distributed learning. Experiments on real hardware show high accuracy, confirming its suitability for constrained environments. Ficco et al. [10] propose a privacy-preserving FL solution enabling local training on constrained IoT devices, avoiding raw data transmission. Llisterri et al. [11] investigate FL on microcontrollers, noting that more frequent training rounds accelerate loss reduction but increase training time and bandwidth usage. Deeper models improve performance but demand more RAM.

Unlike Alpha-FedAvg and FedTinyWolf [4], which address only parts of the FL cycle or rely on simulations, TinyFed runs the full process on real ESP32 devices. It achieves up to 99.5% accuracy and 95% F1-score using < 100 KB of memory, while FedTinyWolf reaches 97.1% with 256 KB SRAM and Alpha-FedAvg, despite 98.9% accuracy, requires external processing. TinyFed thus combines high performance, minimal memory usage, and full local training, reinforcing its practical applicability in constrained IoT scenarios. Recent approaches such as HiveFL [15] enables semi-asynchronous FL with clients that autonomously determine participation, enhancing robustness in Mobile Edge Computing (MEC). Similarly [16], investigate the integration of large models for Mobile Edge Intelligence, a scenario that differs from our constrained-device focus but provides a future perspective.

3 | TinyFed: FL for Constrained IOT Devices

This work addresses the implementation of TinyFed, a lightweight FL framework for resource-constrained IoT devices. The device network is modeled as a graph $G = (V, E)$, where V denotes IoT nodes and E their communication links. Each device $v_i \in V$ holds a local dataset D_i to train a model M_i. Instead of sharing raw data, devices transmit model parameters θ_i to an aggregator, preserving privacy and reducing communication. The aggregator updates the global model $M^{(t+1)}$ by applying a function A, typically using the FedAvg algorithm: $\theta^{(t+1)} = \mathrm{A}\left(\left\{\theta_i^{(t)}\right\}\right)$.

TinyFed orchestrates data collection, local training, and inference on resource-constrained devices $v_i \in V$, managing the exchange of model weights θ_i with an aggregator. Each device minimizes a local loss function $G(\theta_i, D_i)$, while the total communication cost is given by $C(E, T)$, representing data transmitted over network E in T rounds. Reducing $C(E, T)$ is key in bandwidth-limited settings. Unlike cloud-based ML systems, TinyFed enables local training on D_i, reducing latency and enhancing privacy by transmitting only model parameters. This decentralized strategy supports efficient and private learning in resource-constrained IoT environments.

In Figure 1, the MCU used is the ESP32, a microcontroller that meets the minimum requirements for running lightweight programs, including processing, memory, storage, and network connectivity. These features align with the constraints of the system model $G = (V, E)$, making ESP32 suitable for the proposed framework. The figure shows the federated flow of data and models, with steps like data collection, local processing, training, and inference performed on the device. A bidirectional communication channel connects ESP32s to the aggregator node, highlighting their autonomy and collaborative capability in a decentralized setting.

Figure 1 illustrates the TinyFed architecture, including local training and communication with the aggregator. Sensors on each device v_i collect data D_i from the environment. TinyFed extends the capabilities of Micro-Controller Units (MCUs) to train the local model M_i with parameters θ_i, perform inference, transmit weights to the aggregator, and update the model with global parameters $\theta^{(t+1)}$. These tasks run concurrently with data collection. The aggregator applies the function A, computes the global model from received θ_i, and distributes the updated weights back to the devices, ensuring efficient coordination.

Data collection and action tasks are handled by sensors and actuators connected to each device v_i. TinyFed was developed using the Arduino IDE, widely adopted in IoT due to its simplicity, library support, and active community. Programming in C allows lightweight code and dynamic memory management, essential for constrained devices. The integration of libraries such as <FS.h> and <SPIFFS.h> enables efficient local storage of datasets D_i and model parameters θ_i on the ESP32.

To meet the FL requirements on resource-constrained devices v_i, we propose a flexible neural network (NN) model M_i. For this particular showcase, we use the model to detect anomalies in the sensor data D_i, but it can be adjusted to suit other purposes, such as binary classification [17]. The NN model M_i consists of an input layer with 4 neurons, representing the input data: temperature, humidity, luminosity, and voltage collected in D_i. It includes three hidden layers with 16, 8, and 4 neurons, which enable the network to learn and gradually reduce dimensionality, capturing patterns in the input data. Finally, the output layer has 2 neurons, configured for anomaly analysis based on input reconstruction.

The objective of training the model M_i on each device v_i is to allow the network to learn a normal representation of the data D_i and automatically detect substantial deviations from these learned patterns, thereby identifying anomalies. The training process proceeds as follows: first, the data stored on the device D_i is loaded,

24761508, 2025, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/itl2.70061 by Bangladesh Hinari NPL, Wiley Online Library on [12/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

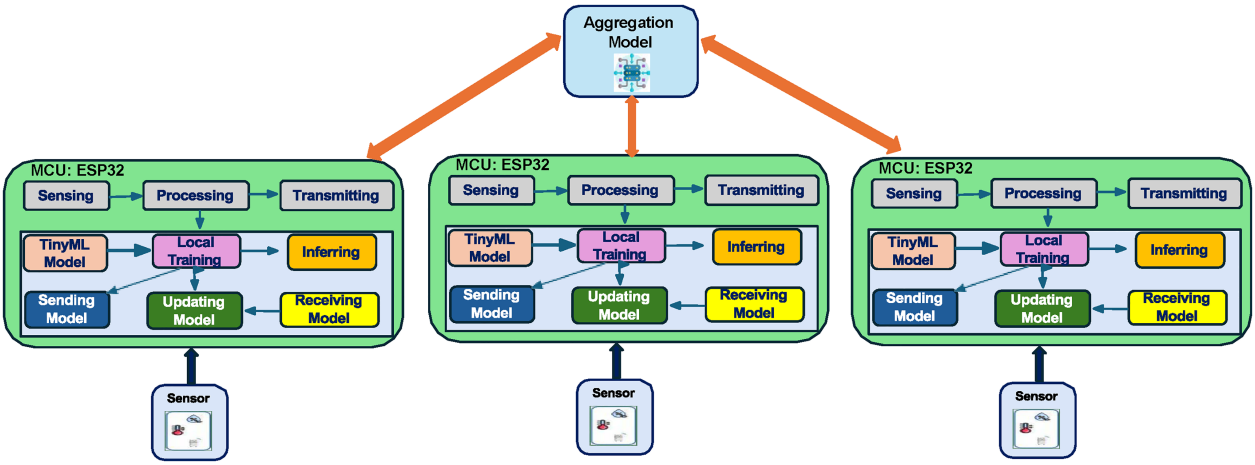


FIGURE 1 | Overview of the distributed TinyML architecture with ESP32 MCUs.

with each sample normalized to ensure that all input values remain within the same range. The model M_i is then trained over multiple epochs, with each epoch covering all samples in the dataset D_i.

During training, a forward pass function calculates the activations of each layer by applying the current model parameters θ_i to each hidden layer and using a sigmoid function to produce the output. This results in reconstructing the input data in the output layers, where the output aims to approximate the original input. The reconstruction error, corresponding to the loss function $G(\theta_i, D_i)$, is then calculated by comparing the network's output with the original input. Finally, the backpropagation function adjusts the weights θ_i in each layer based on the error calculated in the forward pass function. This error is propagated from the output layer back to the first hidden layer, and the weights are updated in each layer according to the defined learning rate.

In FL, after each local training round t, each device v_i shares its updated model parameters $\theta_i^{(t)}$ with the aggregator node to collaborate on global learning. This learning occurs through the model aggregation function A. There are multiple strategies for aggregating models, with FedAvg being one of the most commonly used methods. In our implementation, devices v_i, which have limited resources, send the weights $\theta_i^{(t)}$ to the aggregator server via Message Queuing Telemetry Transport (MQTT) after the end of local training rounds.

After receiving the weights from the various devices, the aggregator node performs the model aggregation function A to compute the updated global model parameters $\theta^{(t+1)}$. In our case, we use a custom FedAvg strategy, which calculates the average of the model weights from all devices. The rationale of the custom FedAvg strategy is to ensure that each device collaboratively trains a global model M while preserving data privacy and reducing communication overhead.

4 | Experiments and Numerical Analysis

In this section, we present the results obtained from the experiments carried out. The objective of the experiments is to evaluate the feasibility of executing the FL process using microcontrollers that can train ML models locally. We configured the test environment using a laptop with an Intel Core i7 8th Gen processor, 16 GB of RAM, Linux Operating System, and 256 GB SSD as a weight aggregator server and an Message Queuing Telemetry Transport (MQTT) server. We set up the federated environment with three ESP32 with 4 MB of storage, expandable to 16 MB, allowing sufficient capacity for our application. In addition, temperature and humidity data were collected using DHT22 sensors. All devices in the experiment were connected through a Wi-fi network.

In the first phase, each ESP32 device collected data, including random anomalies, and stored it locally in CSV files. Then, each device trained a local model on its dataset. After training, devices sent their weights to the aggregator server, which applied the aggregation function and returned the updated weights via MQTT. In the final phase, each device updated its model with the received weights.

To simplify our experiment, we chose a dataset with four features—temperature, humidity, voltage, and luminosity—as our focus is on training and inference across devices. Given that our objective includes evaluating the behavior during data reading, training, and inference, we recognized that using data from an external source could affect model performance. Thus, we implemented a hybrid approach: temperature and humidity data are collected from local sensors, while brightness and voltage values are generated randomly.

5 | Results

Table 1 presents initial information about training execution. We performed 25 training rounds on each device, using datasets representing 70% of the original dataset. Based on the training time evaluation, we observed that each record in the dataset requires 0.04 s. The asymptotic complexity of the time function $T_{\text{total}} = n \times e \times t$ is (n), where n represents the number of samples. Here, T_{total} denotes the total training time, e is the number of epochs, and t is the training time per sample, with $t = 0.04$ s.

24761508, 2025, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/it2.70061 by Bangladesh Hinari NPL, Wiley Online Library on [12/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Characteristics of devices.

Device	Training file (KB)	Samples	Total time (s)	Sample per time
ESP01	33.92	1238	1248.01	0.040065
ESP02	36.46	1331	1333.23	0.040067
ESP03	38.54	1407	1409.16	0.042349

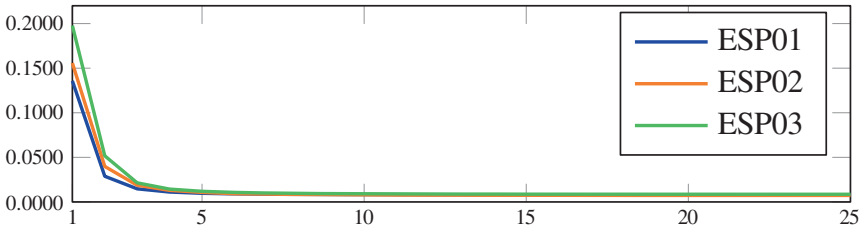


FIGURE 2 | Average mean squared error (MSE).

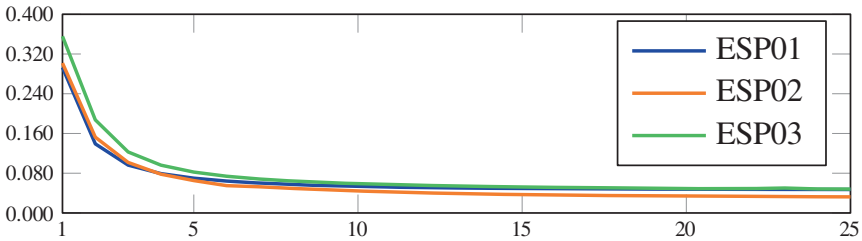


FIGURE 3 | Average mean absolute error (MAE).

We conclude that the number of samples determines the training time. For devices with resource constraints, processing time will directly imply energy consumption. An alternative to reducing training time is to use neuron pruning, removing those whose activation is not significant from the process.

We analyzed the model from the perspective of Mean Squared Error (MSE) and Mean Absolute Error (MAE). Figures 2 and 3 show a steady decline in MSE and MAE values as the model continues training, reflecting its ability to learn and improve its predictions over time. Consistent convergence toward lower values suggests that the model effectively captures data patterns and adapts to minimize errors. Initial analysis indicates that the model achieves its objective and detects anomalies based on reconstruction error, suggesting that it is sensitive to substantial deviations from average data.

Figure 4 shows the accuracy progression for ESP01, ESP02, and ESP03. ESP01 reaches its peak accuracy (98.55%) after seven rounds, while ESP02 and ESP03 converge faster, stabilizing at 99.47% and 99.22% within three rounds. These results reveal differences in convergence speed, with ESP02 and ESP03 adapting more rapidly. The high accuracy across all devices indicates reliable classification performance.

Figure 5 shows the Recall progression for ESP01, ESP02, and ESP03. Recall measures the model's ability to detect all anomalies, with higher values indicating fewer false negatives. ESP01 improves gradually, reaching 77.31% after seven rounds. ESP02 shows consistent growth, peaking at 88.89% in the final round. ESP03 quickly reaches 97.47% in round two, then stabilizes at 86.08% for the rest of the training.

These results highlight the varying rates of learning and convergence among the devices, suggesting differences in their ability to detect anomalies effectively. The overall trends observed in the Recall metrics underscore the model's capacity to adapt and improve in identifying anomalies as training progresses. The graphs illustrate the evolution of accuracy and recall metrics across training rounds for three ESP32 devices. ESP02 and ESP03 reach high performance early, while ESP01 gradually improves, stabilizing after round 7.

Figure 6 shows the F1-score, the harmonic mean of Precision and Recall. As device precision was 100%, F1-score reflects Recall performance. ESP02 and ESP03 quickly achieved high F1-scores, stabilizing at 94.12% and 92.52%, respectively. ESP01 progressed more gradually, rising from 26.67% to 83.93%, indicating a slower learning curve.

Memory is a critical resource in ESP32. To assess usage during training, we monitored the minimum free HEAP memory, as shown in Figure 7. No memory leaks were detected, and fluctuations remained minimal across all devices. ESP01 showed a gradual decrease, stabilizing around 156.63 KB. ESP02 and ESP03 began with 160.35 and 160.75 KB, respectively, both stabilizing near 156.90 KB. The consistent behavior confirms the memory efficiency of local training and the robustness of the implementation in constrained environments.

Table 2 presents the performance metrics for each device, comparing the Local Model (before aggregation) and the Aggregated Model (after the FL process). The federated learning process positively impacted ESP01, which showed the most significant improvement across all metrics. Precision increased slightly from

24761508, 2025, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/itl2.70061 by Bangladesh Hinari NPL, Wiley Online Library on [12/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

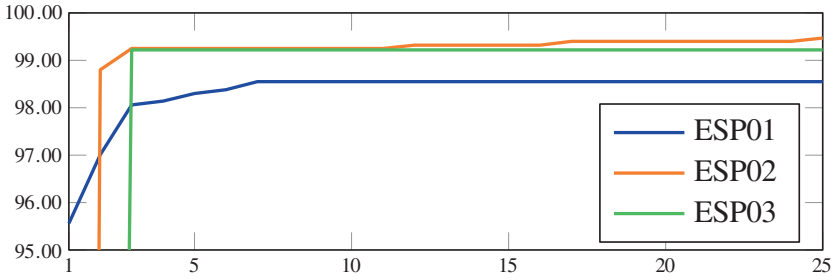


FIGURE 4 | Accuracy.

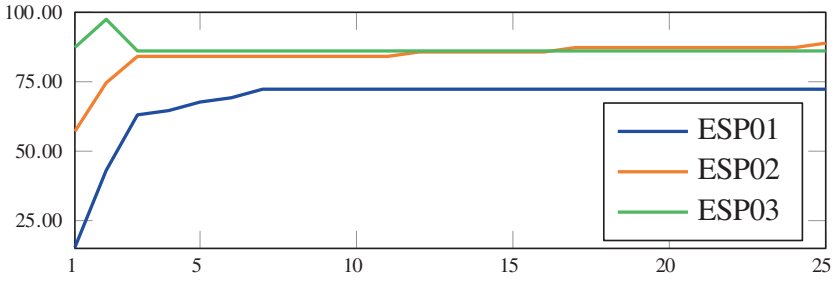


FIGURE 5 | Recall.

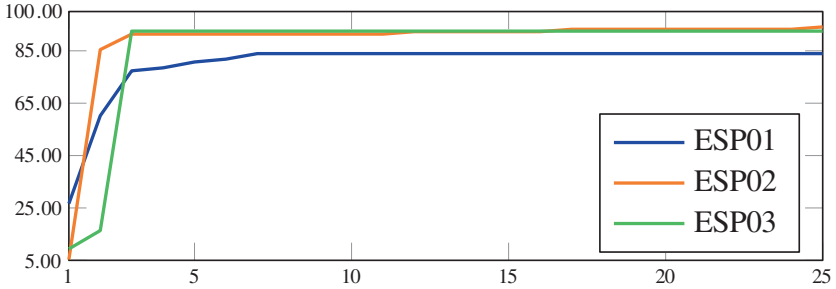


FIGURE 6 | F1-score.

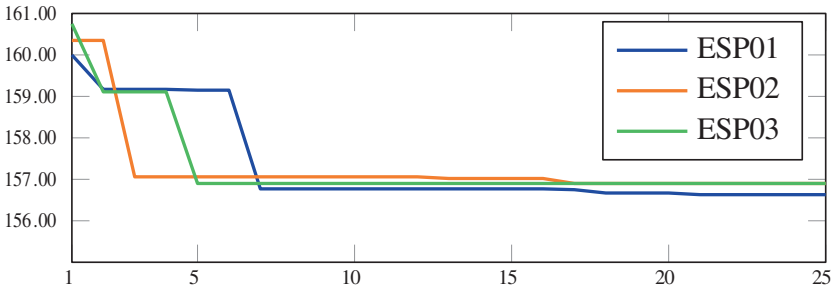


FIGURE 7 | Minimum free HEAP memory (KB).

TABLE 2 | Performance metrics: Local model vs. aggregated model.

	Local model			Aggregated model		
Devices	**Accuracy**	**Recall**	**F1-score**	**Accuracy**	**Recall**	**F1-score**
ESP01	98.55	72.31	83.93	98.79	76.92	86.96
ESP02	99.47	88.89	94.12	99.55	90.48	95.00
ES3654P03	99.22	86.08	92.52	99.22	86.08	92.52

24761508, 2025, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/itl2.70061 by Bangladesh Hinari NPL, Wiley Online Library on [12/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

98.55% to 98.79%, while Recall improved from 72.31% to 76.92%, resulting in a higher F1-score (from 83.93 to 86.96). These results suggest that the aggregated model effectively addressed some of the limitations of the local model for this device.

Improvements were also observed for ESP02, though less pronounced than for ESP01: Accuracy increased from 99.47% to 99.55%, Recall from 88.89% to 90.48%, and F1-score from 94.12% to 95.00%. This suggests that aggregation helped refine an already strong local model. In contrast, ESP03's metrics remained unchanged, indicating that federation had no effect—possibly due to high alignment with the global model or limited data variation. Still, aspects like sample size, learning rate, training epochs, and data diversity warrant further investigation.

6 | Conclusion and Future Works

This work demonstrated that on-device training in such environments is feasible, avoiding reliance on upper layers and reinforcing privacy by transmitting only model weights. For future work, we highlight: (i) scaling to networks with more than 10 devices using hierarchical aggregation and gateway-based coordination; (ii) supporting personalized models per device, aiming for architectural heterogeneity; (iii) integrating asynchronous and fault-tolerant strategies to handle connectivity variations and device availability; (iv) conducting an in-depth analysis of security and privacy in the federated cycle; and (v) evaluating TinyFed across diverse experimental setups, including real-world datasets (e.g., Intel Lab Data, Gas Sensor Array), alternative neural network architectures (e.g., shallower or ReLU-based models), different learning tasks (e.g., classification and time-series prediction), and physical sensors with varying distributions and energy profiles. These directions aim to consolidate TinyFed as a scalable, reliable, and applicable solution for heterogeneous and distributed IoT networks. To the best of our knowledge, TinyFed is the first fully operational FL system on ESP32 microcontrollers reporting end-to-end performance and memory usage metrics across real-world federated rounds.

Acknowledgments

The Article Processing Charge for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614).

Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

References

1. A. Karras, A. Giannaros, C. Karras, K. C. Giotopoulos, D. Tsolis, and S. Sioutas, "Edge Artificial Intelligence in Large-Scale IoT Systems, Applications, and Big Data Infrastructures," in *2023 8th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)* (IEEE, 2023), 1–8.

2. S. Ni, Y. He, L. Chen, Y. Wang, and F. Yu, "A Survey of Edge Computing Resource Allocation Strategies Based on Federated Learning," in *2023 International Conference on Networking and Network Applications (NaNA)* (IEEE Computer Society, 2023), 116–121.

3. P. Warden and D. Situnayake, *TinyML: Machine Learning With TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers* (O'Reilly, 2020).

4. S. Adhikary and S. Dutta, "FedTinyWolf—A Memory Efficient Federated Embedded Learning Mechanism," *IEEE Embedded Systems Letters* 16 (2024): 4, https://doi.org/10.1109/LES.2024.3462638.

5. J. Zheng, J. Xu, H. Du, et al., "Trust Management of Tiny Federated Learning in Internet of Unmanned Aerial Vehicles," *IEEE Internet of Things Journal* 11, no. 12 (2024): 21046–21060, https://doi.org/10.1109/JIOT.2024.3363443.

6. H. Cheng, Q. Chen, Y. Liang, and G. Zhu, "TinyFL: A Lightweight Federated Learning Method With Efficient Memory-and-Communication," in *2023 IEEE Globecom Workshops (GC Wkshps)* (IEEE, 2023), 608–613.

7. C. N. Silva and C. V. S. Prazeres, "Tiny Federated Learning for Constrained Sensors: A Systematic Literature Review," *IEEE Sensors Reviews* 2 (2025): 17–31, https://doi.org/10.1109/SR.2025.3548547.

8. L. Wulfert, N. Asadi, W. Y. Chung, C. Wiede, and A. Grabmaier, "Adaptive Decentralized Federated Gossip Learning for Resource-Constrained IoT Devices," in *DistributedML '23* (Association for Computing Machinery, 2023), 27–33.

9. K. Muhammed and N. Ibraheem, "Alpha-FedAvg: Safeguarding Privacy and Enhancing Forensic Analysis in Federated Learning on Edge Devices," *International Journal of Computing and Digital Systems* 15 (2024): 1391–1403, https://doi.org/10.12785/ijcds/150198.

10. M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, "Federated Learning for IoT Devices: Enhancing TinyML With On-Board Training," *Information Fusion* 104 (2024): 102189, https://doi.org/10.1016/j.inffus.2023.102189.

11. N. Llisterri Giménez, J. Miquel Solé, and F. Freitag, "Embedded Federated Learning Over a LoRa Mesh Network," *Pervasive and Mobile Computing* 93 (2023): 101819, https://doi.org/10.1016/j.pmcj.2023.101819.

12. P. Qi, D. Chiaro, and F. Piccialli, "Small Models, Big Impact: A Review on the Power of Lightweight Federated Learning," *Future Generation Computer Systems* 162 (2025): 107484, https://doi.org/10.1016/j.future.2024.107484.

13. Y. Dai and R. Tan, "FedCFC: On-Device Personalized Federated Learning With Closed-Form Continuous-Time Neural Networks," in *2024 23rd ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)* (IEEE, 2024), 14–26.

14. S. Aatab and F. Freitag, "Integrating a Neural Network Library in Embedded Federated Learning," in *2023 IEEE 6th International Conference on Cloud Computing and Artificial Intelligence: Technologies and Applications (CloudTech)* (IEEE, 2023), 1–7.

15. X. He, H. Huang, B. Xie, et al., "HiveFL: GAN-Empowered Semi-Asynchronous Federated Learning With Self-Determining Clients," *IEEE Transactions on Cognitive Communications and Networking* 11, no. 2 (2025): 805–816, https://doi.org/10.1109/TCCN.2025.3527711.

16. X. He, Y. Jiang, Y. Liu, H. Cui, H. Pan, and Y. Mao, "Transforming 6G Mobile Edge Intelligence With Large Models," *IEEE Network* (2025): 1, https://doi.org/10.1109/MNET.2025.3532486.

17. F. Esmaeili, E. Cassie, H. P. T. Nguyen, N. O. V. Plank, C. P. Unsworth, and A. Wang, "Anomaly Detection for Sensor Signals Utilizing Deep Learning Autoencoder-Based Neural Networks," *Bioengineering* 10, no. 4 (2023): 405, https://doi.org/10.3390/bioengineering10040405.

24761508, 2025, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/itl2.70061 by Bangladesh Hinari NPL, Wiley Online Library on [12/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License